

PROYECTO - Programación III (Elixir)

Contexto

Azar S.A. es una empresa de orden departamental encargada de la explotación, operación y comercialización de juegos de azar en Colombia.

La empresa requiere el desarrollo de un sistema distribuido utilizando **Elixir**, compuesto por múltiples aplicaciones que permitan gestionar sorteos, clientes y apuestas de manera eficiente.

Arquitectura del Sistema

El sistema estará compuesto por **tres componentes principales**:

1. Aplicación servidor central con servidores especializados por sorteo
 2. Aplicación cliente para administradores
 3. Aplicación cliente para jugadores
-

1. Aplicación Servidor

Esta aplicación se ejecuta en la empresa y es el núcleo del sistema.

Funciones principales

- Procesar todas las solicitudes provenientes de los clientes a través de la red.
- Redirigir dichas solicitudes a los servidores especializados según el sorteo correspondiente.
- Gestionar múltiples servidores de sorteos, donde:
 - Cada sorteo tiene un **único servidor asociado**.
 - Cada servidor maneja su información mediante archivos en formato **JSON** (deben incluirse datos de prueba).
- Enviar a los jugadores la información de los **números ganadores por premio**.
- Registrar todas las solicitudes realizadas por las aplicaciones cliente:
 - Mostrar en pantalla (fecha - hora - solicitud y resultado -ok o negado-)
 - Guardar en un archivo de texto (**bitácora/log**).

2. Aplicación Cliente para Administrador

Permite la gestión completa de sorteos y premios.

Gestión de Sorteos

- **Crear sorteo**, especificando:
 - Nombre
 - Fecha
 - Valor del billete completo
 - Cantidad de fracciones
 - Cantidad de billetes (cada uno con número único)
 - **Listar sorteos**:
 - Ordenados por fecha.
 - Mostrar premios asociados (si existen).
 - Si el sorteo ya se realizó:
 - Mostrar número(s) ganador(es).
 - Indicar ganadores por premio (si los hay).
 - **Eliminar sorteo**:
 - Solo si no tiene premios asociados.
 - **Consultar clientes de un sorteo**:
 - Ordenados alfabéticamente.
 - Agrupados en:
 - Compradores de billete completo.
 - Compradores por fracción.
 - **Consultar ingresos por sorteo**
 - **Consultar premios entregados en sorteos pasados**:
 - Mostrar:
 - Premios entregados.
 - Nombres de los ganadores.
 - Dinero recolectado.
 - Calcular:
 - Ganancias o pérdidas.
 - Nota:
 - El valor del premio se divide según la cantidad de fracciones.
 - **Consultar balance de todos los sorteos pasados**:
 - Ganancias o pérdidas por sorteo.
 - Resumen total acumulado.
-

Gestión de Premios

- **Crear premios para un sorteo:**
 - Nombre
 - Valor
 - **Listar premios:**
 - Ordenados por fecha.
 - Agrupados por sorteo.
 - **Eliminar premios:**
 - Solo si el sorteo no tiene clientes asociados.
 - **Actualizar fecha del sistema:**
 - Ejecuta todos los sorteos pendientes hasta la fecha actual.
 - Asigna números ganadores de forma aleatoria (según la cantidad de billetes).
 - Envía las notificaciones correspondientes a los jugadores.
-

3. Aplicación Cliente para Jugadores

Permite a los usuarios interactuar con el sistema y participar en los sorteos.

Funciones principales

- **Registro de usuario:**
 - Nombre
 - Documento
 - Contraseña
 - Tarjeta de crédito (datos simulados)
- **Consultar sorteos disponibles:**
 - Solo aquellos que aún no han sido jugados.
- **Consultar números disponibles:**
 - Billetes completos
 - Fracciones
- **Realizar compras:**
 - Compra de billetes completos.
 - Compra de fracciones.
- **Consultar historial de compras:**
 - Mostrar total gastado.
- **Devolver compras:**
 - Solo si el sorteo aún no se ha realizado.
- **Consultar premios obtenidos**
- **Consultar balance personal:**
 - Diferencia entre dinero gastado y premios obtenidos.

- **Ver notificaciones:**
 - Resultados y mensajes enviados por los servidores de sorteos.
-

Tecnologías (Opcional)

Se permite el uso de herramientas basadas en inteligencia artificial y frameworks como:

- Phoenix Framework

Para el desarrollo de interfaces web que faciliten la interacción con el sistema.

Pero debe comprender el código generado para la sustentación oral.

Notas Finales

- El sistema debe simular un entorno distribuido real.
- Se recomienda aplicar buenas prácticas de concurrencia propias de **Elixir** (procesos, mensajes, supervisores).
- Los datos deben persistirse mediante archivos JSON.
- Se debe garantizar la correcta comunicación entre servidor y clientes.